Why a robot needs a learned model

M18 • 30 Apr 2028 - 04 Jun 2028 • 75 hours

Monthly outcome

Create a reproducible classifier for foot contact with a surface and compare it with a simple rule and logistic regression. Feed it joint and IMU measurements available to the robot, together with leg-motion features. Its outputs are contact probability, a decision of “contact”, “no contact” or “insufficient data”, and the time taken to obtain that decision. Save the data, dataset membership, training configuration, model, error analysis and CPU execution-time measurements.

Learning here addresses a narrow perception task. The classifier helps determine which foot can be trusted when estimating body state. For now, it operates on recordings or alongside a classical algorithm in simulation. A false confirmation of contact can make the estimator treat a slipping or lifted leg as stationary; average accuracy alone therefore does not establish the model's suitability.

What must be ready before starting

Prepare synchronised recordings from previous simulations: time, episode and leg identifiers, joint positions and velocities, body angular velocity, foot position or velocity and an available force estimate. Save simulator contact ground truth only for labelling and evaluation. If recordings are insufficient, perform controlled motions of one leg: free swing, touchdown, stable support, liftoff and slipping. Data collection does not require a complex gait.

Check that you can read CSV and Parquet, plot a time series, explain the mean and standard deviation, calculate a dot product and differentiate squared error. Practise missing skills in the first block. Use your development experience to describe the data format, ensure reproducibility and investigate causes of model errors; there is no need to repeat all Python fundamentals.

What the result establishes

The main outcome is a justified educational experiment. The network is not yet permitted to control a physical robot. The original curriculum mentions G3, and the overall calendar allocates separate time to integration checks. In M18, prepare the data, model and limitations for that stage. Checks are required before any physical use, regardless of the date of G3. In later modules, the classical estimator and controller must also work without machine learning.

The connection to the hand is direct: the same process applies to estimating finger-object contact. However, a positive “touch” label does not mean a “reliable grasp”. Use one main foot-contact task this month; describe the differences in features and errors for transfer to a finger, without requiring a second training project.

75 hours: from data to a decision

Four learning blocks

Hours 1-20: 10 hours of theory and analysis plus 10 hours of practice. Study regression, classification, loss functions, bias and variance. Define features, labels and training, validation and test sets comprising whole episodes. Prepare a dataset description, check synchronisation and establish the first classical baseline method. The deliverable is data that do not expose the correct answer as a model input.

Hours 21-40: 10 hours of theory and 10 hours of practice. Study tensors, automatic differentiation, layers, optimisers, normalisation and overfitting control. Write a small PyTorch training loop and run a compact network. The outcome is a reproducible run, rather than a lengthy architecture search. Change one hypothesis in each experiment; preserve the original configuration unchanged.

Hours 41-60: 10 hours of theory and 10 hours of practice. Study confusion matrices, precision and recall, ROC and PR curves, calibration, class imbalance and data shift. Investigate failures, remove feature groups and test abstention. Choose the contact-confirmation threshold using the validation set; keep test episodes closed until the final comparison.

Hours 61-75: 5 hours of theory and analysis plus 10 hours of practice. Study export, numerical precision, quantisation and the difference between latency and throughput. Measure the complete path from an available measurement to a decision, reproduce the model in a separate process and prepare a computing-platform decision. Total: 35 hours of theory and 40 of practice; unattended waiting for training does not replace learning work.

Scheduling sessions

Divide each 10 hours of theory into five two-hour sessions. Complete practical work in Sunday blocks of 10 hours; each may be divided into four periods with breaks excluded from learning time. Divide the final 5 hours of analysis as 2 + 2 + 1. Numbering denotes the sequence of learning workload. Boundary dates may overlap adjacent modules; rest, holidays and integration blocks remain as in the curriculum, and no hour is counted twice.

At the start of each session, write one testable question, such as “does foot velocity reduce false contact reports at liftoff?”. At the end, save the configuration, plot, conclusion and next step. Limit the active list to three tasks: the current hypothesis, a known data error and the next check. This keeps the module within 75 hours and helps distinguish a model improvement from a change to the experiment itself.

Data, labels and splits without leakage

Task 1. Describe the contact data

For each field, state its name, units, frequency, source, delay and availability on the future robot. Separate the features and ground_truth tables. Features must not contain true contact force, scenario identifier, a future frame or foot coordinates relative to the exact surface when the robot lacks the corresponding sensor. This distinguishes measured quantities from information known only to the simulator. Save data_contract.md and ten rows checked by hand.

Define the positive label from simulator contact and a minimum load chosen for this model. Mark ambiguous touchdown moments uncertain and describe how they are evaluated. Do not adjust the labelling threshold to suit network results. Manually check several intervals before, during and after contact: the label must match the event without a one-packet shift. Report the number of uncertain rows separately.

Task 2. Split by complete experiments

First assign episode_id to complete independent runs. Then create an explicit list of episodes for the train, validation and test directories: for example, 60/20/20 percent of episodes; the number of independent groups matters more than the exact proportions. Adjacent windows from one experiment must stay in the same subset. Group consecutive files from one robot recording together. Separately save an OOD set with new friction, noise or load.

Check that identifiers and source-recording ranges do not overlap. Determine normalisation, missing-value imputation, balancing and feature-selection parameters from the training set only. With few episodes, use grouped cross-validation within it, while retaining separate final test groups. Show the class and condition distributions across subsets. Randomly shuffling time-series rows makes evaluation unjustifiably easy.

Task 3. Check causal availability

For a window of length W, use only measurements timestamped no later than the decision time. Construct the mean, dispersion and maximum velocity over the past window, plus a missing-packet flag. Change future values in the source file: the prediction for the current time must remain unchanged. Deliver a causality test and a plot of window delay. If good metrics require future information, that version is suitable for analysing recordings, but not for online contact estimation.

Start with 6-12 understandable features. Millions of nearly identical frames do not add new conditions. Several episodes with different surfaces and motion phases are more useful than another hour of stationary-foot recordings.

Baseline models and loss functions

What to study before a neural network

Regression predicts a number, such as normal load in newtons; classification predicts a category or contact probability. For an educational comparison, plot squared error against the numerical prediction and binary cross-entropy against the probability of the correct class. Do not conflate objectives: low force-estimation error does not guarantee reliable touchdown detection, particularly near zero.

In the binary task, y is 0 or 1 and p denotes contact probability. The loss is $-[y \log(p) + (1-y) \log(1-p)]$. It penalises confidently wrong answers more strongly. Use a numerically stable implementation operating on logits; avoid manually taking the logarithm of zero or applying sigmoid twice. The resulting probability is used for analysis, not as a multiplier for actuator commands.

Task 4. Two baseline methods for comparison

Build a simple rule from available features: low foot velocity together with a sufficient load estimate, if one exists. Choose thresholds on the validation set. If load is not measured, state that and use kinematics and the commanded phase. The second baseline is logistic regression with normalisation calculated on the training set. Both methods and the network must use identical episodes, inputs and window delay.

Save predictions_baseline.csv, a confusion matrix and errors by episode. Test a stationary leg hanging freely: low velocity alone does not mean contact. Test a slipping support: contact can exist at nonzero velocity. These two cases explain the need for a feature set and why even a reasonable rule is not physical proof of support.

Task 5. Bias, variance and regularisation

Train logistic regression on a small and the full training set, then a compact network with and without regularisation. Compare training and validation losses. High error in both subsets suggests possible problems with features, the model or labels. Low training error with high validation error calls for checks of overfitting and differences in conditions. These observations help investigate a cause but do not establish it automatically.

After the first baseline result, test at most three settings: for example, two regularisation values and one hidden-layer width. Set an overall epoch limit and early stopping based on the validation set. Save a table of hypothesis, change, observation and decision. Claiming improvement requires more than one successful run. Do not repeatedly use the final test set to select an architecture.

PyTorch: training a small model

Minimum topics

Study tensor shape: a row corresponds to a measurement window and a column to a feature. For N examples with D features, the input shape is N × D; the binary model outputs one logit per example. Check dtype, device and target dimensions. Use normalisation and a small multilayer perceptron, MLP, with one or two hidden layers. Complex convolutions, transformers and architecture search are unnecessary for this task.

Task 6. One reproducible run

Create a Dataset from the episode list, a DataLoader, model, loss function, optimiser and separate evaluate function. During training, sequentially zero the gradients, perform the forward pass, calculate loss, perform the backward pass and take an optimiser step. For evaluation, disable gradient accumulation and enable eval mode. Save normalisation, feature order and schema version alongside the weights. Loading must explicitly reject an input-schema mismatch.

Check the loop on a small set of correctly labelled examples: the model should substantially reduce training error. This tests functionality, not generalisation. Then restore the normal data split. Deliberately swap two columns and check that the schema detects the error before prediction: degraded answers alone reveal a format change too late.

Task 7. Gradients and optimisers

For a model with one linear layer, compare one autograd gradient component with a central finite difference for a small weight perturbation. Check the update direction and change in loss after a small step. Then compare SGD and Adam using the same number of epochs, batch size and number of examples. Record the rate of loss reduction and final validation metric; this experiment does not establish the universal superiority of an optimiser.

Study learning rate, weight penalties and a simple step-size reduction schedule. Run one experiment with a deliberately excessive step size to diagnose divergence, then restore a working setting. For each run, save the random-number generator seed, library versions, device, commit, episode list, optimiser settings and best weights. A seed improves repeatability but does not guarantee bitwise agreement across platforms and versions; compare within a pinned environment.

The baseline platform is the CPU. Apple Silicon can use MPS when the required operations are supported, but the MacBook M4 is not a CUDA platform. No GPU purchase is planned for this small tabular network; first measure actual data-preparation and training time.

Performance, confidence and unfamiliar conditions

Metrics must reflect the robot's errors

Let contact be the positive class. Precision = TP/(TP+FP) is the fraction of reported contacts that are correct; recall = TP/(TP+FN) is the fraction of actual contacts detected. A false contact, FP, is particularly harmful to contact odometry, while a missed contact, FN, reduces available information. State the rule for a zero denominator. Additionally evaluate touchdown and liftoff detection delay relative to labelled events, in milliseconds.

Task 8. Thresholds and imbalance

On the validation set, plot a confusion matrix, PR curve and ROC curve. Choose a threshold considering the acceptable number of false contacts in the educational scenario. Define this criterion before the final test; it is not a universal safety limit. Show the number of events and examples in each class. If support dominates the recording, an “always contact” rule may achieve high accuracy: include it to show this metric's limitations.

For rare liftoff events, compare ordinary and weighted losses; calculate class weights on the training set. Recheck calibration after changing example sampling or weights. The test-set distribution must match the stated scenario, without artificial balancing. Show performance for each episode and the average across episodes, so a long recording cannot conceal complete failure on a short one.

Task 9. Calibration and abstention

Group validation probabilities into bins. Compare the mean predicted probability with the actual contact fraction, and show the number of examples in each bin: this produces a reliability diagram. Calculate the Brier score as the mean of $(p-y)^2$. To fit calibration, reserve separate groups from training or validation data; do not use test data. With a small dataset, report estimation uncertainty and the number of observations.

Define reasons for abstention: missing mandatory fields, a stale window, values outside known ranges and insufficient confidence. Range checks detect only some OOD cases; high network probability does not prove that conditions are familiar. Under new friction and noise, measure abstention frequency and remaining errors. On abstention, send unknown to the classical estimator, which applies verified rules and does not automatically accept contact.

Laboratory: contact recognition

Specification for contact_model_v1

Replay leg recordings by timestamp. First check the schema and data freshness, then construct features solely from measurements available at that moment. The model outputs a probability, the threshold block produces a status, and a separate check compares it with contact ground truth. No actuator commands are sent here. On one plot, show contact ground truth, prediction, unknown, foot velocity and rejection reason; mark touchdown, liftoff and slipping with lines.

In the input-data description, list files, hashes, robot model, simulator, durations, classes and conditions. Specify window, features, seed, model, epoch limit, thresholds and abstention rules in the configuration. Save predictions for every valid time instant, episode metrics and timing measurements. Account for rejected rows too: their number, reasons and proportion of all measurements must remain visible, otherwise removing difficult cases artificially improves the metric.

Task 10. Final comparison on the test set

After choosing the configuration, pin the commit and open the test groups. Evaluate the rule, logistic regression and chosen MLP in the same way. Compare precision, recall, event delay, unknown fraction and processing time. Perform paired comparisons on identical episodes. To estimate variability, resample whole episodes rather than neighbouring frames. If there are too few episodes for a confident conclusion, state that.

The curriculum criterion is improvement over a simple baseline on held-out data. Before testing, choose the primary metric and a false-contact limit. If the network does not win, retain the negative result and mark the criterion unmet; collect new data in the next iteration. Reproducible evaluation is still useful: it may provide sound grounds for choosing the simpler model.

Task 11. Feature ablation

Remove commanded-phase information, kinematic velocity and IMU features in turn. Train each version on the same training set and select settings on the same validation set. Use a separate analysis set, preserving the independence of the final test. Compare errors during liftoff, slipping and a stationary leg in the air. Formulate a hypothesis about the value of each information source and save two plots of failed episodes.

Finally, reproduce the work from a clean directory using the README. Run training, performance evaluation and timing through separate commands, so checking a saved model does not require retraining.

Tests: eight ways to go wrong

Data and behaviour checks

1. **Adjacent windows in different subsets.** Prepare a deliberately invalid list in which one episode_id appears in train and test. The check must reject it before training. On diagnostic data, compare random row splitting with grouped splitting. Explain the inflated estimate through correlation between neighbouring observations. Save an automated overlap report and both estimates, identifying the split method.

2. **A stationary leg in the air.** Replay a halted swing without contact. A velocity-only rule may produce FP; test the network and combined baseline under the same conditions. Record the duration of false contact and explain which measurements can distinguish the cases. If the sensors are insufficient, use unknown.

3. **A slipping support.** Use contact with nonzero tangential velocity. The contact label remains positive; “non-slipping support” is a different task. Check that the analysis does not call all slipping frames mislabelled. Provide a separate support-quality indicator for the future estimator, without substituting it for the current objective.

4. **Missing data and delay.** Delete a sequence of packets and delay one channel. Record the age of the oldest measurement used. After the limit specified in the configuration, the decision must become unknown. Check instants before and after the threshold, and recovery after a full window has accumulated from data already received.

Model and execution checks

5. **New load and friction.** Test physically valid OOD episodes outside the training ranges. Compare errors with the ordinary test set and measure the fraction of OOD cases detected. Describe a confidently incorrect answer and the remaining risk: perfect detection of unfamiliar conditions cannot be expected.

6. **NaN and a broken schema.** Supply a non-numeric value, reordered columns and a different velocity unit. NaN and schema-version mismatches must be rejected before the network. Check unit errors using explicit metadata and ranges: not every unit change is detectable from a single number. Record the precise reason code.

7. **Repeatability.** Repeat the fixed configuration in the same environment, then compare three seed values. Report variability in the primary metric and training time, including all runs. Evaluate agreement across platforms using a numerical tolerance chosen beforehand.

8. **Export and a slow consumer.** Compare the original model with the saved or exported model on identical inputs, then artificially slow processing. Reading a result later must not change its age. Measure the number of missed processing deadlines and the observer's response; save the actual timestamp sequence.

Session sequence: hours 1-40

Block 1: task and data

Hours 1-2: draw the chain “leg motion → measurement → feature → label → decision”. Study how contact differs from stable support and permission to transfer weight. List errors that the experiment must detect. Hours 3-4: study regression, classification, loss functions and overfitting; calculate two losses and a confusion matrix for ten predictions by hand. Do not move to PyTorch until the values in this table are clear.

Hours 5-6: study grouped splitting and causal windows. On paper, split six short experiments so that one scenario is not fragmented into neighbouring rows in different sets. Hours 7-8: describe features, units and sensor availability. Hours 9-10: establish the baseline models, primary metric and test protocol. These five sessions provide the required 10 hours of analysis.

Hours 11-20 - first laboratory: 2 hours for extraction and manual label checks; 2 for episode lists and leakage checks; 2 for causal feature preparation; 2 for the rule and logistic regression; 2 for failure plots and recording the baseline result. If importing data is difficult, reduce the number of scenarios while retaining both classes and independent episodes. Calculate normalisation after splitting the data, and preserve the test set.

Block 2: training a small model

Hours 21-22: tensors, shapes and types; a dimension table from input to loss. Hours 23-24: autograd and the gradient step; manual derivative and finite-difference checks. Hours 25-26: Dataset, DataLoader, train/eval modes and disabling gradients for evaluation. Hours 27-28: SGD, Adam, learning rate, regularisation and early stopping. Hours 29-30: saving the model, configuration, seed values and experiment log.

Hours 31-40 - second laboratory: 2 hours for the training loop and tiny-dataset check; 2 for the first full run; 2 for debugging gradients and data preparation; 2 for two bounded hypothesis comparisons; 2 for restoring saved weights and checking reproducibility. Retain unsuccessful runs. If the network does not learn, check labels, dimensions, gradients and normalisation before changing model size.

By the end of hour 40, retain the original classical method unchanged and one working learned-model configuration. A runs directory, metrics in CSV and a README with the reproduction command suffice. Every number must be traceable to specific data and settings; an additional interface, cloud logging and complex automation are optional.

Session sequence: hours 41-75

Block 3: testing hypotheses

Hours 41-42: precision, recall and PR; check denominators and the meaning of the positive class by hand. Hours 43-44: compare ROC and PR under class imbalance, and select a threshold on validation data. Hours 45-46: calibration and its diagram, with examples of confidently wrong answers. Hours 47-48: unfamiliar OOD conditions - noise, load, friction and time shifts. Hours 49-50: feature-ablation plan and error inventory.

Hours 51-60 - third laboratory: 2 hours for confusion matrices and thresholds; 2 for calibration and unknown; 2 for missing data and shifts; 2 for limited feature ablation; 2 for the final comparison and analysis of the two worst episodes. Identify the dataset for every number. Once the final test is opened, a new setting requires an explicitly declared next iteration and new independent evaluation.

Block 4: measurements when applying the model

Hours 61-62: study export and the difference between a weights format and an executable graph; check that the entire preprocessor transfers with the model. Hours 63-64: study batch size, single-decision latency, warm-up and memory. Hour 65: describe remaining deployment limitations and what the later G3 check will require. These are the final 5 theory hours, divided as 2 + 2 + 1.

Hours 66-75 - fourth laboratory: 2 hours for loading, export and output comparison; 2 for CPU measurements; 2 for reproducing the complete measurement-to-decision path; 2 for reproduction from a clean directory; 2 for the report and computing-platform choice. Measure after warm-up, separately from cold start. State repeat count, batch=1, thread count, CPU model, power mode and competing workload.

Task 12. Is an accelerator needed?

Separate total latency into window accumulation, preparation, prediction computation and result transfer. Measure p50, p95 and p99 - values below which 50, 95 and 99 percent of observations lie - plus the maximum and deadline-miss count. These measurements do not prove an upper time bound. Throughput in examples/s at a large batch size does not replace latency for one fresh measurement. A GPU can accelerate the network while adding transfer overhead.

Justify any need for Jetson and CUDA using latency, memory, energy consumption and supported operations. Purchasing hardware and authorising learned walking control are considered at the separate G3 checkpoint. If the CPU meets requirements, an accelerator need not be purchased. Otherwise, first identify the latency source and reduce the model. Test quantisation separately: repeat accuracy, calibration and OOD evaluation. Reduced memory use does not guarantee acceleration.

Acceptance checks and handover

Materials that support the result

Create a contact_model_v1 directory containing README, data_contract, manifests, configs, baseline, model, reports and measurements. In the README, explain how to obtain the source logs and provide train, evaluate, replay and profile commands. The report must state the objective, measured features, label provenance, operating limits, environment versions and device specifications. Keep exact configurations even for rejected models; otherwise, the reasons for preferring the final choice cannot be reconstructed.

Criterion 1: no overlapping episodes, future frames or preprocessing fitted on test data; automated and manual checks are attached. Criterion 2: the baseline is compared with the network on a set kept closed until the final evaluation, using a preselected metric and FP limit; if there is no improvement, the criterion remains unmet. Criterion 3: repetition with the same config and seed gives explainably similar results in one environment. Criterion 4: the complete data path is measured, including processing before and after the network.

Criterion 5: OOD errors, unknown, missing data and delays are shown; the classical branch remains available. Criterion 6: a review of “data, model and limits of use” is prepared for integration testing. A saved weights file does not mean readiness for control. Before physical use, sensors, the actuator chain, independent limits and stopping are checked separately; the calendar placement of G3 does not replace these actions.

Self-check without notes

Explain: 1) why row-based splitting inflates performance; 2) why normalisation must not be fitted on all data; 3) how logits differ from probabilities; 4) why a stationary foot may not touch the floor; 5) why contact and absence of slipping are different objectives; 6) how confidently confirming a false contact affects body-state estimation. Support your answers with your own plots.

Also answer: 7) how calibration differs from classification; 8) why ROC can conceal the importance of rare errors; 9) where window-accumulation delay arises; 10) what a seed provides; 11) why export requires retesting; 12) which measurements justify a GPU. If an answer depends on an unknown robot characteristic, record its clarification as a requirement for the next stage.

If time is short

If time is short, remove additional architectures, then the MPS/GPU comparison, then the practical quantisation experiment while retaining its theory. Keep the CPU, small network, simple features and grouped data split. Retain baselines, leakage checks, OOD, full-latency measurement and reproducibility. Hand the probability and unknown formats to M19 with their limitations. Contact estimation for the classical gait must have a verified path without the network.

Assigned reading: learning from robot data

Read the specified extracts before the corresponding tasks. Core sources explain principles; references assist implementation. Reading is included in the 75 hours. Sources may be in English; prepare your explanations and report in English.

Core reading

1. Gareth James, Daniela Witten, Trevor Hastie, Robert Tibshirani, Jonathan Taylor, An Introduction to Statistical Learning with Applications in Python, 2023. Chapters 2, 4, 5: authors' materials. Statistical Learning - error and generalisation; Classification - logistic regression; Resampling Methods - validation and cross-validation. For Tasks 1-5: justify episode splitting and baseline selection; not all laboratory exercises need to be completed.

2. Aston Zhang, Zachary Lipton, Mu Li, Alexander Smola, Dive into Deep Learning. §2.5 Automatic Differentiation; §3.7 Weight Decay; §5.1-5.3 (Multilayer Perceptrons; Implementation; Forward/Backward Propagation). For Tasks 6-7: one small MLP, gradients and weight updates; perform the exercises using your own contact features.

Reference for an unclear result

3. Ian Goodfellow, Yoshua Bengio, Aaron Courville, Deep Learning, 2016. Ch. 5 Machine Learning Basics; Ch. 7 Regularization for Deep Learning. Find explanations of model complexity, overfitting and your chosen regularisation for Task 5. Ch. 8 Optimization for Training Deep Models - only the selected optimiser for Task 7, without reading the whole chapter.

Documentation needed for the experiment

PyTorch: Learn the Basics - Tensors, DataLoaders, Autograd, Optimization, Save/Load for Tasks 6-7.
scikit-learn: Common pitfalls - Data leakage and preprocessing for Tasks 1-3; fit transformations only on the training set.

scikit-learn: Probability calibration - Calibration curves and Calibrating a classifier for Task 9. Explain FP, FN and the chosen threshold using your own confusion matrix; supplement accuracy with error analysis.

First read the core source for the topic, then test your understanding on the task. Consult the third book for explanations of a specific difficulty; reading all three textbooks in full is not required this month. CPU measurements and conditions for accelerator use are described on the preparation page.

Fundamental concepts in M18

Feature. A measured or computed quantity available at decision time; future values are inadmissible.

Label. The target answer for training; here it denotes contact under the adopted rule, not absence of slipping.

Training set (train). Data used to fit model parameters and feature transformations; these parameters are learned only from this set.

Validation set (validation). Separate data for selecting hyperparameters and the threshold; used before final evaluation.

Test set (test). Separate episodes for final evaluation; tuning on them compromises the evaluation's independence.

Data leakage. Information used in training or evaluation that is unavailable during application; for example, adjacent frames placed in different subsets.

Standardisation. Subtracting a mean and dividing by a scale; statistics are obtained from the training set and fixed.

Logistic regression. A sigmoid applied to a linear combination of features, yielding a class probability; here, for binary contact.

Loss function. A measure of disagreement between prediction and label that training minimises; it is not the same as operational usefulness.

Gradient. A vector of loss derivatives with respect to parameters; it indicates the direction of local increase for weight updates.

Backpropagation. Calculating gradients using the chain rule; an optimiser changes weights in a separate step.

Regularisation. A constraint or penalty against fitting peculiarities of training data; its strength is selected on validation data.

Overfitting. Improvement on training data by fitting their specific features, accompanied by deterioration on new episodes from the same distribution.

Precision. The fraction of true contacts among predicted contacts: TP/(TP+FP); it reflects false confirmations.

Recall. The fraction of detected contacts among all positive labels: TP/(TP+FN); it reflects misses at the chosen threshold.

Probability calibration. Agreement between probabilities and frequencies on independent data; accurate classification does not guarantee it.

Decision threshold. A boundary for class selection or abstention; set according to error costs before the final test.

Distribution shift. A change in conditions relative to training; new materials can reduce performance while confidence stays unchanged.

Ablation. Removing a feature or component under an unchanged protocol to assess its contribution and possible leakage.

Inference latency. The time required for prediction; total latency also includes preparation, transfer and subsequent processing.

Equipment and software for M18

Use existing equipment

Use the MacBook Pro M4, previous synchronised leg and IMU recordings, and the simulator. Perform training and timing primarily on the CPU in macOS ARM. If necessary, export ROS recordings from Ubuntu 24.04 ARM64 in UTM; preserve episode_id, time, units and the separation of features from ground truth.

No mandatory purchases

The required classifier needs no Jetson, NVIDIA GPU, new camera or new contact sensor. First measure the entire data-processing path of a small MLP and logistic regression. Obtain contact labels through the adopted simulator protocol and store them separately from model input features.

Install now: a separate Python environment

In the uv/venv already used on macOS ARM, add torch, scikit-learn and JupyterLab if absent; reuse NumPy/Matplotlib/pytest. Choose the Python version for compatibility with the pinned packages, and save the lock and manifest. Instructions: PyTorch Start Locally, scikit-learn install, Jupyter install.

Select CPU execution; CUDA is not installed for Mac M4. Add pandas and pyarrow only if needed to read input tables; the established loader suffices for CSV. Do not install training dependencies in ROS's system Python. Transfer data between environments using files in an agreed format.

Installation and reproducibility checks

Import torch and sklearn, check CPU tensor operations, perform forward and backward passes through a small network and take one optimiser step. Train LogisticRegression on a small training set, then save and reload it. After restarting the notebook, reproduce the saved episode split and classification metrics.

Record seed values, versions, thread count and chosen device; repeat a short run following PyTorch Reproducibility. For Task 12, measure warmed-up repetitions and cold start, including features and subsequent processing: PyTorch Benchmark.

Optional, after measurement

You may compare the built-in MPS with CPU after checking operation availability; retain the CPU result. An accelerator purchase is considered only for a demonstrated computational limitation and a separate G3 decision. G3 on 27.11-10.12.2028 is still ahead; M18 does not complete it.

Installation and checks use time already allocated to setup within the original 75 hours. This month's result is a model and measurements from recordings or simulation. The classical algorithm must operate independently of machine learning.

Motion fundamentals: a model learned from data

Required analysis: what regression learns about a leg or finger joint

From physical law to learned parameters

Before Tasks 4 and 7, explain the distinction between an unknown parameter and an unknown state. Consider a revolute joint in a vertical plane without contact: q is the angle from the downward vertical in rad, qdot is velocity in rad/s and qdd is acceleration in rad/s². Positive torque τ acts in the direction of increasing q. Geometry and axis direction have already been checked.

I·qdd + b·qdot + k·sin(q) = τ. I is the moment of inertia about the axis, kg·m²; b is the viscous-friction coefficient, N·m·s/rad; k=m·g·l_c is the known gravity coefficient, N·m; m is link mass in kg, g=9.81 m/s² and l_c is the distance from the axis to its COM in m. Assume a rigid link, constant I,b and no backlash. Take τ from the educational simulation; a commanded servo angle and supply current are not measured joint torque.

Move the known gravitational torque term: y=τ−k·sin(q), φ=(qdd,qdot), θ=(I,b). This gives y=φ·θ: motion is nonlinear in q, but the regression is linear in the unknown I,b. Minimise the sum of squared residuals in N²·m². Conditions I>0 and b≥0 express positive kinetic energy and power dissipation b·qdot²; low error alone does not ensure them.

A small calculation: parameters and a new pose

Use the educational value k=0.20 N·m. Two independent samples at q=0 are (qdot,qdd,τ)=(1,2,0.14) and (2,−1,0.03), in the stated SI units. Thus 2I+b=0.14 and −I+2b=0.03, giving **I=0.05 kg·m², b=0.04 N·m·s/rad**. For a separate sample q=π/6, qdot=0.5, qdd=−2, predict τ=−0.10+0.02+0.10=**0.02 N·m**.

Two exercises with the baseline model

A. In the existing notebook, solve this system using NumPy and save id_check.csv containing the inputs, units and prediction for the third row. Check the answer by substitution and power balance: in the second row, input power 0.03×2=0.06 W equals the kinetic-energy rate −0.10 W plus friction losses 0.16 W. This checks the calculation on known data; generalisation is evaluated on separate episodes, as in Tasks 1-5.

B. Set qdd=2·qdot in every row. The feature matrix has rank 1: only 2I+b can be identified, not both parameters separately. More identical motions do not resolve this. In a free-motion recording, check synchronisation of τ and q; differentiation amplifies noise. For a current estimate, use only measurements already received. Contact forces and simulator ground truth remain in labelling and evaluation; M18's rules still apply.

Hours: 1 hour from session 3-4 on regression and loss functions, and 1 hour from the first laboratory, 11-20, allocated to baseline analysis. The physical example replaces the generic regression example. Retain the main classifier, data split, OOD and timing measurements. The total remains 35+40=75 hours.

Selected reading: Russ Tedrake, System Identification: “Estimating inertial parameters (and friction)”, “Lumped parameters for the simple pendulum”, “Equation error vs simulation error”. Explain why a small equation residual does not guarantee an accurate prediction over a long interval.